

Documentazione Tecnica Progetto

"Falling Bombs"

Questa documentazione fornisce un'analisi dettagliata di ogni classe componente il videogioco *Falling Bombs*, un sistema di difesa arcade sviluppato in Java Swing. Il software si distingue per un'architettura fortemente orientata al multi-threading, dove ogni entità di gioco opera su un flusso di esecuzione indipendente.

1. Analisi delle Classi Core

Game.java

La classe Game funge da entry-point dell'intera applicazione. La sua responsabilità principale è il coordinamento dell'interfaccia utente ad alto livello e la gestione degli stati del gioco.

- **Gestione Finestra:** Inizializza il JFrame principale, impostando dimensioni (600x800) e centratura.
- **Navigazione:** Utilizza un CardLayout per permettere lo switch fluido tra il Menu principale e il pannello di gioco MyPanel.
- **Ciclo di Vita:** Il metodo startNewGame() si occupa di istanziare un nuovo pannello di gioco, azzerando punteggi e liste, e avvia l'entità Terrorista.
- **Post-Game:** Implementa un timer di 3 secondi dopo il "Game Over" prima di riportare l'utente al menu principale, garantendo la leggibilità del punteggio finale.

Menu.java

Classe dedicata alla visualizzazione della schermata di benvenuto.

- **Layout:** Sfrutta il GridBagLayout per garantire che il titolo e i pulsanti rimangano centrati indipendentemente dai ridimensionamenti.
- **Interattività:** Riceve un ActionListener nel costruttore per gestire l'evento di inizio partita in modo disaccoppiato dalla logica UI.

MyPanel.java

Rappresenta il cuore pulsante del gioco. Questa classe gestisce sia il rendering grafico che la logica di collisione globale.

- **Strutture Dati:** Utilizza CopyOnWriteArrayList per tutte le liste di entità (bombe, proiettili, esplosioni). Questa scelta è critica per la thread-safety: permette di iterare sulle liste per il disegno mentre altri thread aggiungono o rimuovono oggetti.
- **Controllo Collisioni:** Il metodo controllareCollisioni() verifica le intersezioni tra i rettangoli (hitbox) delle entità. Gestisce logiche complesse come l'interazione tra esplosioni e bombe o l'attivazione dei PowerUp.
- **Rendering:** Il metodo paintComponent disegna lo sfondo, il cannone (giocatore), tutte le entità attive e le scritte di interfaccia (Score, Game Over, modalità CIWS).
- **Input:** Implementa un KeyAdapter per captare i movimenti e il comando di fuoco (tasto Spazio).

2. Analisi delle Entità di Gioco (Thread-Based)

Giocatore.java

Modella il difensore alla base dello schermo.

- **Movimento:** Gestisce due variabili booleane (destra, sinistra) aggiornate dagli input di MyPanel. Il thread interno aggiorna la posizione X ogni 100ms.
- **Limiti:** Garantisce che il cannone non esca dai margini laterali dello schermo.

Bomba.java

L'ostacolo principale del gioco.

- **Logica di Caduta:** Ogni bomba incrementa la propria coordinata Y in base alla speed assegnata casualmente alla creazione.
- **Condizione di Sconfitta:** Se la bomba supera la maxY (il fondo dello schermo), istanzia un'EsplosioneNaturale e invoca il metodo gameOver() nel pannello principale.

Esplosivo.java

Una variante avanzata della bomba.

- **Comportamento:** Si muove come una bomba standard, ma possiede un flag che la identifica come materiale infiammabile. Se intercettata da un proiettile, genera un'area di danno circolare (Esplosione) che può innescare reazioni a catena.

Proiettile.java

Entità generata dal giocatore per neutralizzare le minacce.

- **Movimento:** Viaggia esclusivamente verso l'alto (Y decrescente).
- **Gestione Memoria:** Si autodistrugge (rimuovendosi dalla lista) non appena esce dal bordo superiore per liberare risorse.

PowerUp.java

Oggetto bonus che cade dal cielo.

- **Effetto:** Se colpito, non incrementa il punteggio ma attiva un timestamp nel MyPanel. Questo permette al giocatore di entrare in modalità "Fuoco Rapido" (CIWS), riducendo il delay tra i colpi da 500ms a 10ms per una durata definita.

3. Gestori di Sistema ed Effetti

Terrorista.java

Rappresenta l'intelligenza artificiale (IA) nemica o lo spawner.

- **Thread Generativo:** Il thread osama esegue un loop infinito (finché non è Game Over) che genera casualmente tre tipi di oggetti: Bombe (80% di probabilità), Esplosivi (10%) o PowerUp (10%).
- **Variabilità:** Ogni entità viene creata con una posizione X e una velocità di caduta randomiche, aumentando la difficoltà.

Esplosione.java

Gestisce l'effetto area generato dalla distruzione di un Esplosivo.

- **Area d'Impatto:** Il metodo getBounds() restituisce un rettangolo molto più grande dell'entità originale (300x300), permettendo di eliminare più bombe contemporaneamente.
- **Timer Interno:** Un thread dedicato monitora la durata dell'esplosione, rimuovendola dopo 20 cicli per pulire lo schermo.

EsplosioneNaturale.java

Classe puramente estetica.

- **Scopo:** Viene visualizzata quando una bomba viene distrutta o tocca terra. A differenza della classe Esplosione, non ha hitbox e non può distruggere altre entità, servendo solo come feedback visivo per l'utente.
-